

02



合 计 方 法

>>> 分摊分析

- **分摊分析:** 执行一系列数据结构操作所需的时间是对所执行的所有操作的平均值。
- 分摊分析与平均案例分析的不同之处在于不涉及概率。
- 摊销分析可确保在最坏情况下每项操作的平均性能。
- 在运行分摊分析过程中存储的代价是用于分析，并没有实际意义。不能出现在最终的代码中。

2.1 合计方法

- 在聚合方法中，我们考虑对所有 n 种可能的情况的最差时间进行加和，那么平均时间就是总时间除以 n 。
- 注意，即使操作序列中有不同种类的操作分摊代价也是对每个操作来说的。

■ 栈操作

Push(S, x) 将元素 x 入栈 S 。

Pop(S) 将栈顶元素出栈 S 并返回。

- 上述每个操作耗时 $O(1)$ ，假设每个操作用时为1。由 n 个入栈出栈操作组成的序列总耗时为 n ，那么序列实际总耗时为 $\Theta(n)$ 。

- MultiPop(S, k) 弹出至多k个栈顶元素.

MultiPop(S, k)

1 **while** not StackEmpty(S) and $k \neq 0$ **do**

2 Pop(S)

3 $k \leftarrow k - 1$

MultiPop(S, k) 平均耗时是多少?

top $\rightarrow$ 23

17

6

39

10

47

MultiPop(S, 4)

top $\rightarrow$ 10

47

MultiPop(S, 7)

- 让我们分析一个初始空堆栈上的 n 个Push, Pop和MultiPop操作的序列。
- 任何堆栈操作的最坏情况时间是 $O(n)$ ，因此 n 个操作的序列花费 $O(n^2)$ 。
- 虽然单个MultiPop操作可能很昂贵，但是在最初为空的堆栈上的任何 n 个Push, Pop和MultiPop操作序列的成本最多为 $O(n)$ 。操作的平均成本是 $O(n) / n = O(1)$ 。

- 二进制计数器的加和
- 考虑实现从0开始向上计数的 k 位二进制计数器的问题。
- 该操作的运行时间与翻转的位数成比例。

Increment(A)

1 $i \leftarrow 0$

2 **while** $i < n$ and $A[i] = 1$ **do**

3 $A[i] \leftarrow 0$

4 $i \leftarrow i + 1$

5 **if** $i < n$ **then**

6 $A[i] \leftarrow 1$

Counter value	A[7]	A[6]	A[5]	A[4]	A[3]	A[2]	A[1]	A[0]	total cost
0	0	0	0	0	0	0	0	0	0
1	0	0	0	0	0	0	0	1	1
2	0	0	0	0	0	0	1	0	3
3	0	0	0	0	0	0	1	1	4
4	0	0	0	0	0	1	0	0	7
5	0	0	0	0	0	1	0	1	8
6	0	0	0	0	0	1	1	0	10
7	0	0	0	0	0	1	1	1	11
8	0	0	0	0	1	0	0	0	15
9	0	0	0	0	1	0	0	1	16
10	0	0	0	0	1	0	1	0	18
11	0	0	0	0	1	0	1	1	19
12	0	0	0	0	1	1	0	0	22
13	0	0	0	0	1	1	0	1	23
14	0	0	0	0	1	1	1	0	25
15	0	0	0	0	1	1	1	1	26
16	0	0	0	1	0	0	0	0	31

- 在最坏的情况下，单次执行Increment需要时间 $O(k)$ 而数组 A 包含全1。因此，在最坏情况下，在初始零计数器上的 n 个递增运算序列需要时间 $O(nk)$ 。
- 我们可以收紧我们的分析，通过观察每次调用Increment时不是所有的位都会翻转，从而为 n 次递增序列产生最坏情况下的 $O(n)$ 成本。

- 总翻转次数

$$\sum_{i=0}^{\lfloor \lg n \rfloor} \left\lfloor \frac{n}{2^i} \right\rfloor < n \sum_{i=0}^{\infty} \frac{1}{2^i} \\ = 2n$$

- 因此，对于初始零计数器上的 n 个递增运算序列的最坏情况时间是 $O(n)$ 。每个操作的平均成本，以及每个操作的摊余成本，是 $O(n)/n = O(1)$ 。

谢谢大家

